

CSC 2400-002: Design of Algorithms (Spring 2026)

Tennessee Tech University

Instructor: Prantar Ghosh

Programming Assignment Checkpoint 2

Name: Jocelyn Smith, Logan Wolford, Killian Miller

Credit Statement: We worked only within our group members



Conclusions

Due Date: 03/31/26

In this checkpoint, we worked with a dataset of 50 e-commerce items, where each item included a price, rating, and number of purchases. The goal was to implement and compare three sorting algorithms. These sorting algorithms are Merge Sort, Quick Sort using Lomuto's partition, and Quick Sort using Hoare's partition. We sorted the data based on two features: price and number of purchases, in both increasing (low-to-high) and decreasing (high-to-low) order. The sorted results were then written to separate output files. We also measured the runtime of each algorithm while sorting different sizes of the dataset to get a better understanding of their performance. From a theoretical standpoint, all three algorithms have an average time complexity of $O(n \log n)$. Merge Sort's worst case runs in $O(n \log n)$ while both Quick Sort algorithm's worst cases run in $O(n^2)$. Merge Sort always runs in $O(n \log n)$. So, it is pretty much always going to have a consistent performance regardless of what the input happens to be. While Quick Sort is the opposite. Quick sort is much slower with its worst-case complexity. This happens whenever the pivot is always the largest or smallest element, so the splits are unbalanced. Despite this, Quick Sort is generally faster because it has less overhead compared to Merge Sort. It also becomes progressively less and less likely for the worst case to happen as the dataset grows.

In our experiments with the checkpoint, the runtimes of the algorithms were very similar when sorting, but they also had very clear differences. Merge Sort was always the slowest by a very solid amount, and Hoare's Quick Sort was always the fastest by a clear margin. Lomuto's Quick Sort was consistently right in the middle of Merge Sort and Hoare's Quick Sort. We believe that there are a couple reasons for this result. Merge Sort was likely the slowest because of how we were creating new arrays to hold everything in and dynamically allocating memory like that likely increased its runtime. Hoare's and Lomuto's algorithms are very interesting as while they are both Quick Sort, the different partition methods do create a noticeable difference. This difference in time is likely created due to the fact that Hoare's algorithm starts with its pointers on both sides of the array and because of that, it is swapping two elements at a time while Lomuto's algorithm goes through the elements one by one and divides them according to their sizes relative to the selected partition. This means that Hoare's Quick Sort will almost always perform fewer swaps and with therefore be faster on average as a result.

In conclusion, this checkpoint displayed that while our theoretical analysis is useful for predicting how algorithms will grow, our actual performance depends on implementation and input size, and especially how the data is distributed. In the relatively unlikely event that either of the quicksort algorithms do hit their worst-case performances, Merge sort will beat both of them hands down. Because of that, we believe that Merge sort is best used in cases where consistency matters more than anything else, and The Quick Sort algorithms, most notably Hoare's algorithm, are best used when speed is what matters more than anything else.